

Introdução

Contexto

Áreas:

- Existem basicamente três áreas centrais da Teoria da Computação: **Autômatos, Computabilidade e Complexidade.**

Áreas que estão interligadas pela questão:

- Quais são as capacidades e limitações fundamentais dos computadores?

Conceitos Básicos

Alfabeto:

- Um **alfabeto** é um conjunto finito não vazio de símbolos ou caracteres. O alfabeto sobre o qual as cadeias são definidas pode variar com a aplicação.
- Portanto:
 - Um conjunto infinito não é um alfabeto;
 - o conjunto vazio é um alfabeto.
 - os membros do alfabeto são os símbolos do alfabeto;
 - Geralmente, usamos letras gregas maiúsculas Σ e Γ para designar alfabetos e a escrita máquina de escrever para símbolos de um alfabeto.
 - Γ é gama
 - Σ é sigma
- Exemplos:

- $\Sigma_1 = \{0, 1\}$ e $\Gamma_1 = \{a, b\}$ (Binário)
- $\Sigma_2 = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$ (26 símbolos)
- $\Sigma_3 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ (10 símbolos)
- $\Gamma_2 = \{1, 5, 10, 25, 50, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O\}$ (20 símbolos)
- $\Gamma_3 = \{\#, BD, AED, SO, LP, SD, ELD, AOC\}$ (8 símbolos)

○

Palavra, cadeia ou sentença:

- Uma **palavra** ou **cadeia sobre um alfabeto** é uma **sequência finita de símbolos (do alfabeto) justapostos**.
- Dado um alfabeto Σ , uma **cadeia** $\omega \in \Sigma$ é uma **sequência** $\omega_1 \omega_2 \dots \omega_n$, onde $\omega_i \in \Sigma$ para cada $1 \leq i \leq n$, ou seja, é uma sequência finita de símbolos.
- Portanto:
 - **Uma cadeia sem símbolos é uma palavra válida.**
 - ϵ ou λ **denotam a cadeia vazia ou palavra vazia.**
 - ϵ é epsilon
 - λ é lambda
- Exemplos:

- $\Sigma_1 = \{0, 1\}$ (Binário)
 - 01001 é uma palavra sobre Σ_1 .
- $\Sigma_2 = \{a, b, c, \dots, z\}$
 - abracadabra
 - abacaxi
 - apple
 - wyyzywwzz
 - zyd
- $\Sigma_3 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
 - 1
 - 33
 - 00031
 - 00

Comprimento:

- O comprimento de uma palavra é a quantidade de símbolos que compõem a palavra.
- Assim:
 - Se w é uma cadeia sobre um alfabeto, o comprimento de w , escrito $|w|$, é o número de símbolos que ela tem.
 - Se w **tem comprimento** n , podemos **escrever** $w = w_1 w_2 \dots w_n$, onde cada $w_i \in \Sigma$.
- Exemplos:

Exemplos

- $\Sigma_2 = \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\}$
 - $|abacaxi| = 7$
 - $|apple| = 5$
 - $|wyyzywwzz| = 9$
 - $|zyd| = 3$
- $\Sigma_3 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
 - $|1| = 1$
 - $|33| = 2$
 - $|00031| = 5$
 - $|00| = 2$

○

Número de Ocorrências:

- Se $x \in \Sigma$ e w é uma cadeia sobre Σ , denotamos por $|w|_x$ a **quantidade de vezes que o símbolo x aparece em w** .
- Exemplo:

Seja α uma cadeia sobre Σ_2 e β uma cadeia sobre Γ_3 .

$\alpha = abacaxi$

$\beta = ELD\#AOC\#SO$

$|\alpha|_a = 3$

$|\beta|_{\#} = 2$ e $|\beta|_{AOC} = 1$

○

Palavra vazia:

- A **palavra vazia** é uma palavra de **comprimento zero**, ou seja, uma palavra que não contém símbolos!
- Representação:
 - ϵ (geralmente utilizaremos esta para palavra vazia.)
 - λ (geralmente utilizaremos como representação de um símbolo “coringa”, sem tamanho)

Reverso:

- O reverso de w , escrito w^R , é a cadeia obtida escrevendo w em ordem inversa, isto é, $w_n w_{n-1} \dots w_1$
- Exemplos:

$$\begin{aligned} \cdot w &= \text{abacate} \\ \cdot w^R &= \text{etacaba} \end{aligned}$$

○

Concatenação:

- Uma palavra pode ser **concatenada/somada/anexada** a outra **palavra**, formando uma **nova palavra**.
- Se temos a **cadeia x de comprimento m** e a **cadeia y de comprimento n** , a **concatenação de x e y** , escrito como xy , é a **cadeia obtida concatenando-se y ao final de x** , como em $x1 \dots xmy1 \dots yn$.
- Para concatenar uma cadeia com **ela mesma muitas vezes usamos a notação com expoente**. A concatenação de x com ela mesma k vezes.

$$\begin{aligned} \overbrace{xx \dots x}^k &= x^k \\ x^k &= xx^{k-1} \\ x^0 &= \epsilon \end{aligned}$$

○

- Exemplos:

$$\begin{aligned} \cdot \Sigma_2 &= \{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z\} \\ \cdot abacaxi \cdot apple &= abacaxiapple \\ \cdot apple \cdot abacaxi &= appleabacaxi \\ \cdot abacaxi \cdot &= abacaxi \\ \cdot \epsilon \cdot apple &= apple \\ \cdot \epsilon \cdot \epsilon &= \\ \cdot \Sigma_3 &= \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} \\ \cdot 1 \cdot 33 &= 133 \\ \cdot 00031 \cdot 0 &= 000310 \end{aligned}$$

○

- Seja $\alpha = aba$ e $\beta = cac$:
 - $\alpha\beta = abacac$ e $\beta\alpha = cacaba$
 - $\alpha^3 = abaabaaba$
 - $\alpha^2\beta^3 = abaabacaccaccac$

○

Composição de palavras:

- Uma **palavra pode ser composta por outras** (consequência das operações de concatenação).
- Formas de composição:
 - **Sufixo**
 - **Prefixo**
 - **Subpalavra**
- Algumas convenções:
 - **Representaremos palavras como “variáveis” nomeados com letras gregas em minúsculo: α, β, γ .**

Prefixo:

- Uma **palavra α é prefixo** de outra **palavra β** se é possível **escrever β como $\alpha \cdot \gamma$**
- Exemplo:

- **ab é prefixo de abc**
 - $\beta = abc$
 - $\alpha = ab$
 - $\gamma = c$
 - Logo: $\beta = \alpha \cdot \gamma$

○

- ϵ é prefixo de abc
 - $\beta = abc$
 - $\alpha = \epsilon$
 - $\gamma = abc$
 - Logo: $\beta = \alpha \cdot \gamma$
 - $\alpha = abc$
 - $\gamma = \epsilon$
 - $\beta = \alpha \cdot \gamma$

Sufixo:

- Uma **palavra** α é **sufixo** de outra **palavra** β se é possível **escrever** β **como** $\gamma \cdot \alpha$
- Exemplo:

- bd é sufixo de abc
 - $\beta = abc$
 - $\alpha = bc$
 - $\gamma = a$
 - Logo: $\beta = \gamma \cdot \alpha$
- é bc não bd

- ϵ é sufixo de abc
 - $\beta = abc$
 - $\alpha = \epsilon$
 - $\gamma = abc$
 - Logo: $\beta = \gamma \cdot \alpha$

Subpalavra:

- Uma **palavra** α é **subpalavra** de outra **palavra** β se é possível **escrever** β **como** $\gamma \cdot \alpha \cdot \delta$
- Exemplo:

- - bc é subpalavra de $abcd$
 - $\beta = abcd$
 - $\gamma = a$
 - $\alpha = bc$
 - $\delta = d$
 - Logo: $\beta = \gamma \cdot \alpha \cdot \delta$
- - ϵ é subpalavra de $abcd$
 - $\beta = abcd$
 - $\gamma = a$
 - $\alpha = \epsilon$
 - $\delta = bcd$
 - Logo: $\beta = \gamma \cdot \alpha \cdot \delta$

Palavra vazia:

- A palavra vazia é prefixo, sufixo e subpalavra de qualquer outra palavra.

Ordenação Lexicográfica:

- A **ordenação lexicográfica** de **cadeias** é a mesma que a ordenação familiar do dicionário, **exceto que as cadeias curtas precedem as cadeias mais longas**.
- Exemplo:

- O ordenação lexicográfica de todas as cadeias sobre o alfabeto $\Sigma_1 = \{0, 1\}$.

$(\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots)$

Uma **linguagem** é um conjunto de cadeias.

Autômatos Finitos Determinísticos

Linguagens Regulares

A Hierarquia de Chomsky é uma classificação de Linguagens, e apresenta os mecanismos que são seus reconhecedores e geradores.

Tipo	Linguagem	Gramática	Reconhecedor
[0]	Linguagens Recursivamente Enumeráveis	Gramáticas irrestritas	Máquina de Turing
[1]	Linguagens Sensíveis ao Contexto	Gramáticas Sensíveis ao Contexto	Máquina de Turing com Fita Limitada
[2]	Linguagens Livres de Contexto	Gramáticas Livres de Contexto	Autômatos com Pilha
[3]	Linguagens Regulares	Gramáticas Regulares	Autômato Finito

- De acordo com a **Hierarquia de Chomsky**, as linguagens regulares constituem a classe de linguagens mais simples, sendo possível desenvolver algoritmos de **reconhecimento**, de **geração** e **conversão** entre formalismos de pouca complexidade, de grande eficiência e de fácil implementação.
- As **Linguagens Regulares** ou do Tipo 3 podem ser estudadas usando os seguintes formalismos, pelos quais elas podem ser geradas ou reconhecidas.
 - **Autômatos Finitos**: Formalismo operacional ou **reconhecedor**, sendo um sistema de estados finitos. **Autômatos Finitos Determinísticos e Autômatos Finitos Não Determinísticos**.
- **Conjuntos Regulares (Expressões Regulares)**: Formalismo denotacional, também considerado **gerador**, pois se pode **inferir como construir todas as palavras da correspondente linguagem**. Definido a partir de **conjuntos** (linguagens) básicos e de **operações** de **concatenação** e **união**.
- **Gramáticas Regulares**: Trata-se de um formalismo axiomático ou **gerador** o qual, como o nome indica, é uma **gramática**, mas com **restrições da forma das regras de produção**.

- **Linguagens Regulares** possuem fortes limitações de expressividade. Por exemplo, uma linguagem que possua **duplo balanceamento não é uma linguagem regular**. [Parênteses balanceados, cada um aberto tenha o seu correspondente, aninhamento de IFs e ELSEs]
- Portanto o tratamento de expressões aritméticas comuns não pode ser realizado usando formalismo regular. A **maioria das linguagens de programação de propósito gerais** como **Pascal, C, Java, etc...** são **não regulares**.
- Um exemplo típico e simples de utilização de linguagens regulares é a **Análise Léxica**. **Outros exemplos**: Sistemas de animação, hipertextos e hipermídias
- **Definição de Linguagem Regular**:
 - Uma **Linguagem** é chamada de uma **Linguagem Regular** se algum **Autômato Finito** a reconhece.

Autômatos Finitos

- **Autômatos Finitos** ou **Máquinas de Estados Finitos** são uma forma matemática de descrever tipos particulares de algoritmos (ou “máquinas”).
- Podem ser **utilizados para descrever o processo de reconhecimento de padrões em cadeias de entrada**, e assim podem ser utilizados para construir **sistemas de varredura**.
- Há uma forte relação entre **autômatos finitos e expressões regulares**
 - Autômatos possibilitam a implementação de **reconhecedores** de linguagens **geradas** por gramáticas ou **especificadas** por Expressões Regulares.
- **Definição**:
 - Um **Autômato Finito Determinístico (AFD)** é uma 5-upla
 - $M = (Q, \Sigma, \delta, q_0, F)$, onde:
 1. Q é um conjunto finito de estados,
 2. Σ é um conjunto finito chamado o alfabeto de entrada,
 3. δ é a função de transição, tal que $\delta : Q \times \Sigma \rightarrow Q$,
 4. q_0 é o estado inicial, tal que $q_0 \in Q$
 5. $F \subseteq Q$ é o conjunto de estados de aceitação.

Exemplo

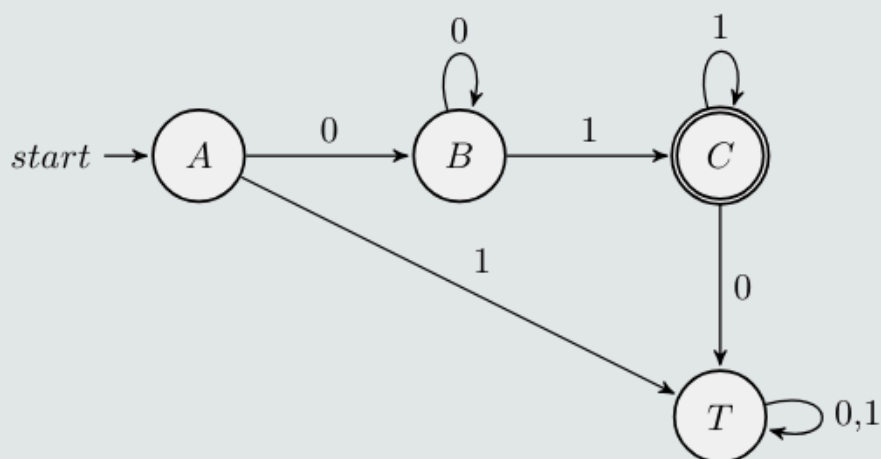
$$L_1 = \{0^n 1^m \mid n \geq 1 \wedge m \geq 1\}$$

Como poderíamos construir um autômato para reconhecer palavras que pertençam à Linguagem L_1 . Em L_1 as palavras são formadas por n ocorrências de 0s seguidas de m ocorrências de 1s. Como $n \geq 1 \wedge m \geq 1$, as palavras devem ter pelo menos uma ocorrência de 0 seguida por um 1.

Exemplo

$$L_1 = \{0^n 1^m \mid n \geq 1 \wedge m \geq 1\}$$

Desenho do Autômato



- T de TRAP, usamos para proteger de erros!

Tabela de Transições

Estado	0	1
A	B	T
B	B	C
C*	T	C
T	T	T

Exemplo: Processamento de uma palavra

Estado atual	Símbolo atual	Estado	
		Resultante	Palavra restante
A	0	$\delta(A, 0) = B$	0111
B	0	$\delta(B, 0) = B$	111
B	1	$\delta(B, 1) = C$	11
C	1	$\delta(C, 1) = C$	1
C	1	$\delta(C, 1) = C$	ϵ

Função de transição estendida

Dado o AFD $A = \langle Q, \Sigma, q_0, \delta, F \rangle$, a Função de Transição Estendida δ é definida como $\delta : Q \times \Sigma^* \rightarrow Q$, tal que:

- $\forall s \in Q, \delta(s, \epsilon) = s$
- $(\forall s \in Q) \wedge (\forall a \in \Sigma) \wedge (\forall x \in \Sigma^* \mid |x| = 1 \wedge x_1 = a), \delta(s, x) = \delta(s, a)$
- $(\forall s \in Q) \wedge (\forall a \in \Sigma) \wedge (\forall x \in \Sigma^*), \delta(s, ax) = \delta(\delta(s, a), x)$

Função de transição estendida

Dado o AFD $A = \langle Q, \Sigma, q_0, \delta, F \rangle$, a Função de Transição Estendida δ é definida como $\delta : Q \times \Sigma^* \rightarrow Q$, tal que:

- $\forall s \in Q, \delta(s, \epsilon) = s$
 - Para todo estado s do autômato finito, a função de transição estendida a partir desse estado s e para a palavra vazia (ϵ) terá como resultado o próprio estado s .
- $(\forall s \in Q) \wedge (\forall a \in \Sigma) \wedge (\forall x \in \Sigma^* \mid |x| = 1 \wedge x_1 = a), \delta(s, x) = \delta(s, a)$
- $(\forall s \in Q) \wedge (\forall a \in \Sigma) \wedge (\forall x \in \Sigma^*), \delta(s, ax) = \delta(\delta(s, a), x)$

Função de transição estendida

- $(\forall s \in Q) \wedge (\forall a \in \Sigma) \wedge (\forall x \in \Sigma^* \mid |x| = 1 \wedge x_1 = a), \delta(s, x) = \delta(s, a)$
 - Para todo estado s do autômato finito, para todo símbolo a do alfabeto de entrada e para toda palavra x de tamanho 1 e que tenha o símbolo a , a função de transição estendida a partir do estado s e para a palavra x terá o mesmo resultado da função de transição (normal) aplicada para o estado s e o símbolo a do alfabeto de entrada.
- $(\forall s \in Q) \wedge (\forall a \in \Sigma) \wedge (\forall x \in \Sigma^*), \delta(s, ax) = \delta(\delta(s, a), x)$

Função de transição estendida

- $(\forall s \in Q) \wedge (\forall a \in \Sigma) \wedge (\forall w \in \Sigma^*) \wedge (w = ax), \delta(s, ax) = \delta(\delta(s, a), x)$
 - Para todo estado s do autômato finito, para todo símbolo do alfabeto de entrada e para toda palavra w considerando o alfabeto de entrada (sendo que a palavra w possui o símbolo a como prefixo de uma palavra x), temos que, para a função de transição estendida a partir do estado s e para a palavra w , o resultado será igual ao da função de transição (normal) a partir do estado s e com o primeiro símbolo a que prefixa a palavra w em questão.

AFD

Decisão sobre a pertinência de uma palavra

Pertinência

Dado o AFD $A = \langle Q, \Sigma, q_0, \delta, F \rangle$ e uma palavra $x \in \Sigma^*$, temos que:

- A aceita x se e somente se (sse) $\delta(S_0, x) \in F$.
- A rejeita x se e somente se (sse) $\delta(S_0, x) \notin F$.

- onde está S_0 deveria estar q_0

Linguagem reconhecida por um AFD

Linguagem de AFD

Dado o AFD $A = \langle Q, \Sigma, q_0, \delta, F \rangle$, define-se a linguagem $L(A)$, ou seja, a linguagem reconhecida pelo autômato A , como:

- $L(A) = \{x \in \Sigma^* \mid \delta(S_0, x) \in F\}$

Configuração de um AFD

Configuração de um AFD

A configuração de um autômato representa a situação atual da execução do autômato. O que permite:

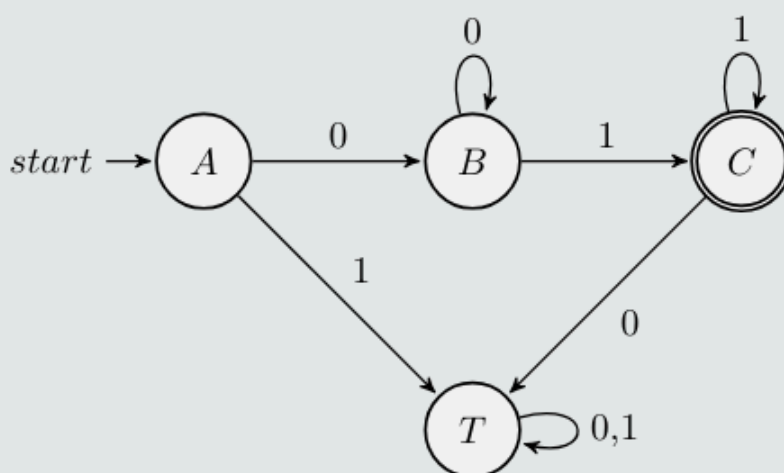
- continuar a execução em outro momento,
- estabelecer as configurações possíveis do autômato.

Para um **AFD**, é suficiente conhecer:

- estado atual
- sequência de símbolos que faltam ser processados , **Não é necessário** saber quais estados que foram alcançados anteriormente.
- Isto não interfere na aplicação da função de transição e, por conseguinte, no funcionamento do autômato.

Exemplo

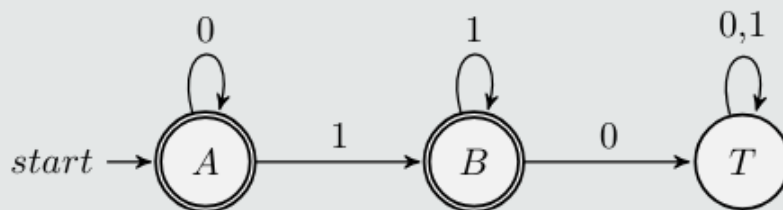
Por exemplo, para o Autômato que reconhece L_2 a configuração 000000B111111000 indica que a execução está no estado B antes de consumir o primeiro 1 da entrada.



Outro Exemplo

$$L_2 = \{0^n 1^m \mid n \geq 0 \wedge m \geq 0\}$$

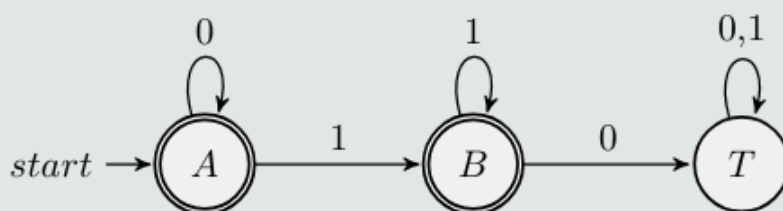
Desenho do Autômato



Descrevendo Autômatos

Descreva o Autômato:

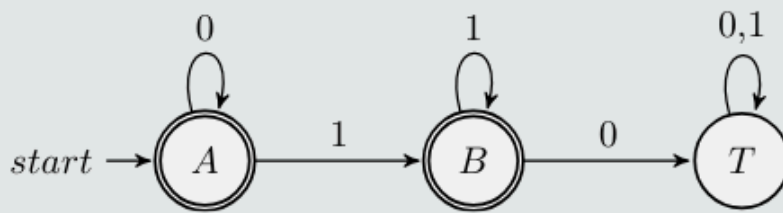
Desenho do Autômato



Podemos descrever M_2 escrevendo $M_2 = (Q, \Sigma, \delta, q_0, F)$:

1. $Q = ?$
2. $\Sigma = ?$
3. $\delta = ?$
4. $q_0 = ?$
5. $F = ?$

Desenho do Autômato



Podemos descrever M_2 escrevendo $M_2 = (Q, \Sigma, \delta, q_0, F)$:

1. $Q = \{A, B, T\}$
2. $\Sigma = \{0, 1\}$
3. $\delta = \{(A, 0) \rightarrow A, (A, 1) \rightarrow B, (B, 0) \rightarrow T, (B, 1) \rightarrow B, (T, 0) \rightarrow T, (T, 1) \rightarrow T\}$
4. $q_0 = A$
5. $F = \{A, B\}$

Autômatos Finitos Não Determinísticos

Introdução e Conceitos

Definição:

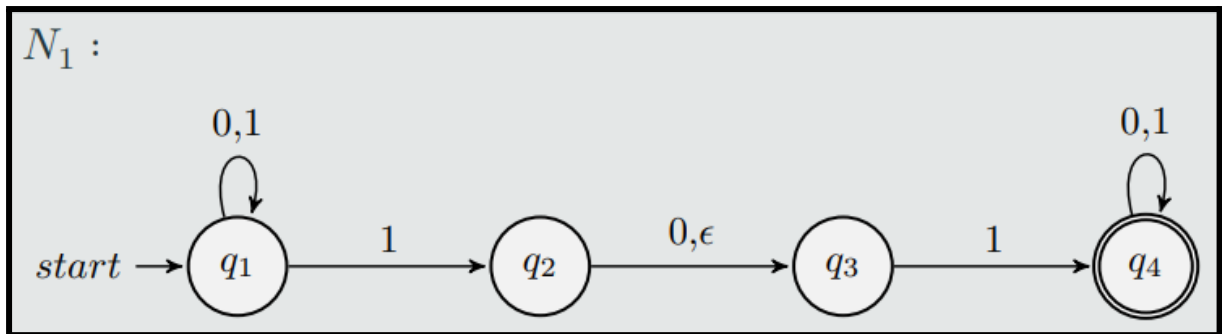
- Um **Autômato Finito Não Determinístico (AFND)** é uma quintupla
- $A = (Q, \Sigma, \delta, Q_0, F)$ onde:
 1. Q é um **conjunto finito de estados**
 2. Σ é o **alfabeto de entrada, finito**
 3. δ é a **função de transição de estados**, tal que $\delta : Q \times \Sigma^* \rightarrow \mathcal{A}(Q)$
 - a. $\Sigma^* = \Sigma \cup \{\epsilon\}$
 - b. $\mathcal{A}(Q) = 2^Q$: Conjunto das partes de Q , todos os subconjuntos de Q .
 4. Q_0 é o **conjunto de estados iniciais**, tal que $Q_0 \subseteq Q$,
 5. F é o **conjunto de estados finais**, tal que $F \subseteq Q$.

Diferenças com relação a AFD:

- Agora é um **conjunto de estados iniciais**.

- Função de transição de estados, para um mesmo símbolo, pode resultar em vários estados.
- Função de transição de estados pode ser para ϵ .
- O símbolo λ também pode ser utilizado para representar símbolo vazio.

Exemplo:



- A descrição formal de N_1 é $(Q, \Sigma, \delta, Q_0, F)$, onde:

1. $Q = \{q_1, q_2, q_3, q_4\}$
2. $\Sigma = \{0, 1\}$
3. δ é dado como:

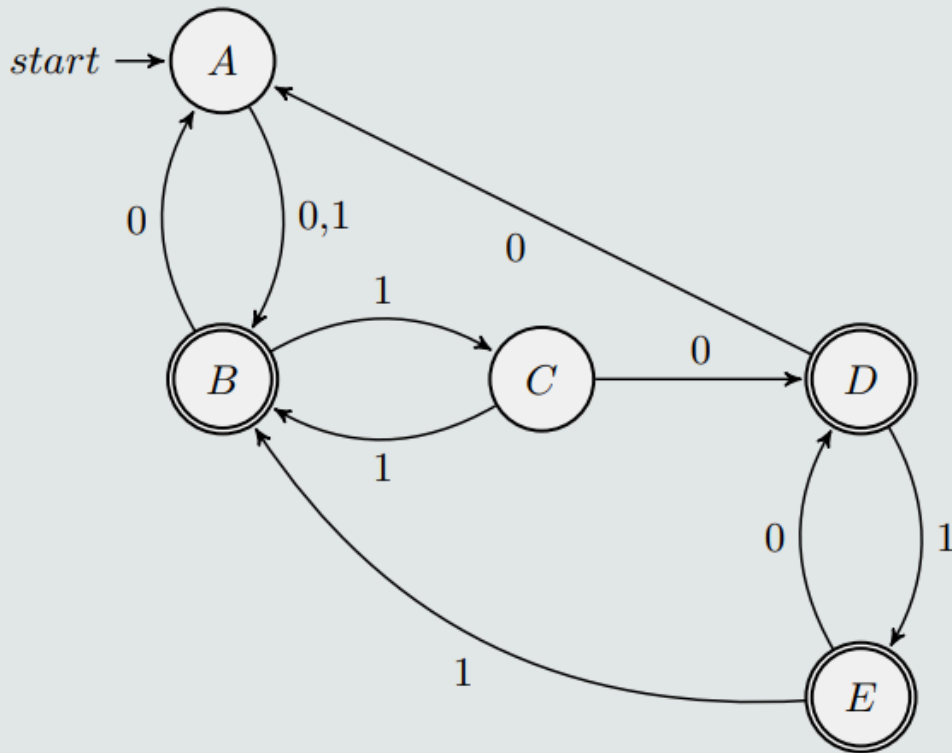
	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

a.

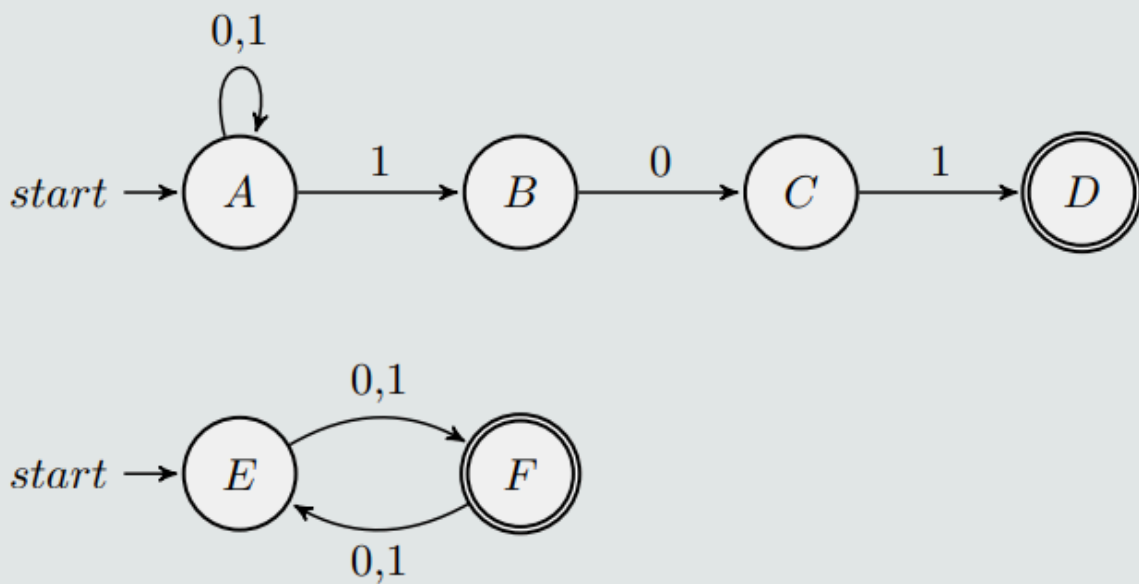
4. $Q_0 = \{q_1\}$ é o estado inicial, e
5. $F = \{q_4\}$

Mais exemplos:

$$L = \{xa \in \{0,1\}^* \mid a = 101 \vee |xa| \bmod 2 = 1\}$$



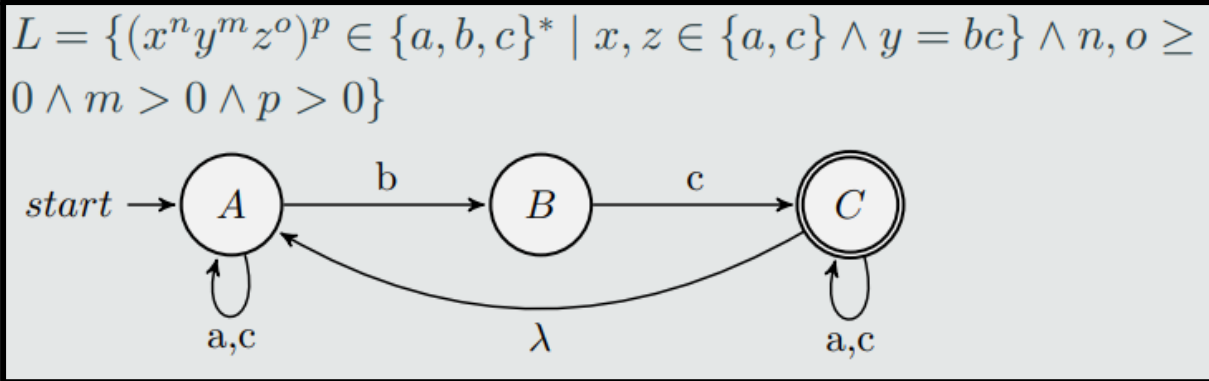
$$L = \{xa \in \{0,1\}^* \mid a = 101 \vee |xa| \bmod 2 = 1\}$$



Transições λ

Transição que ocorre sem que nenhum símbolo de entrada seja processado (ou, melhor, **é processado o símbolo λ**).

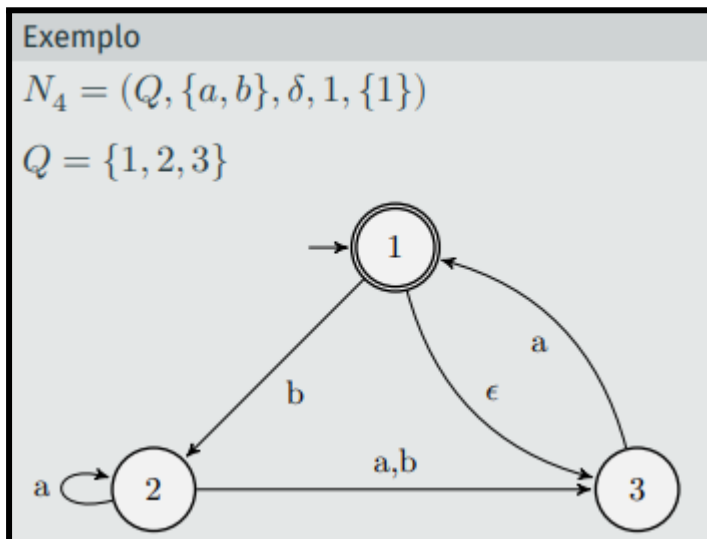
Exemplo:



Equivalência de AFNDs e AFDs

- Se L é um conjunto/linguagem aceito por um **Autômato Finito Não-Determinístico**, então existe um **Autômato Finito Determinístico** que aceita L

Transformação de AFND para AFD com setas ϵ ou λ



- Para construir um AFD D equivalente a N_4 :
 - Determinar os estados de D .**
 - N_4 tem três estados, $Q = \{1, 2, 3\}$. Assim, construímos D com 8 estados, um para cada subconjunto de estados de N_4
 - $Q' = P(Q)$

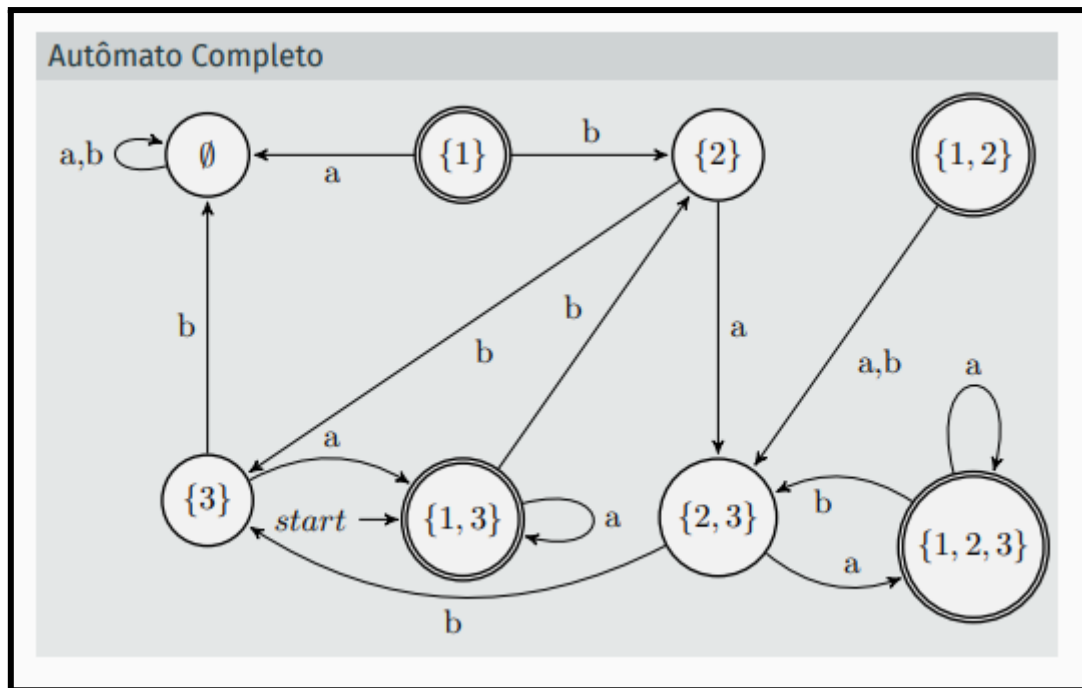
- i. Rotulamos cada um dos estados de D com um subconjunto correspondente.
- ii. $Q' = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$

2. Determinar os estados inicial e de aceitação de D

- a. O estado inicial é $E(\{1\})$, “o conjunto de estados que são atingíveis a partir do estado 1 por setas ϵ , mais o próprio 1.”
- b. Uma seta ϵ vai de 1 para 3; portanto $E(\{1\}) = \{1, 3\}$. Logo, o estado inicial será o $\{1, 3\}$.
- c. Os novos estados de aceitação são aqueles estados que contêm algum estado de aceitação de $N4$. Assim,
 - i. $F' = \{\{1\}, \{1, 2\}, \{1, 3\}, \{1, 2, 3\}\}$.

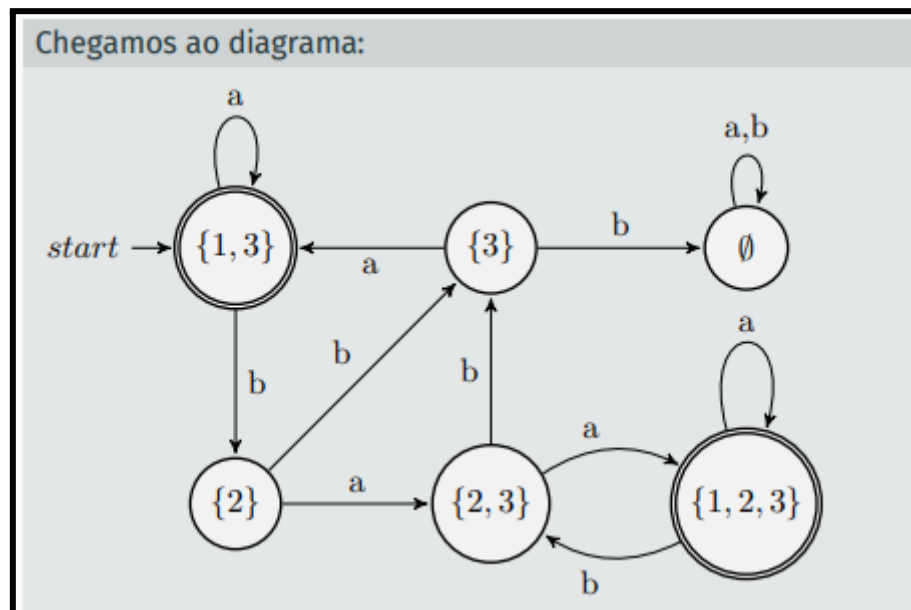
3. Determinar a função de transição de D

- a. Cada um dos estados de D vai para um lugar dada a entrada a e um lugar quando a entrada é b .
- b. Em D , o estado $\{2\}$ vai para $\{2, 3\}$ na entrada a , porque em $N4$ o estado 2 vai para ambos 2 e 3 na entrada a e não podemos ir mais longe a partir de 2 ou 3 ao longo de setas ϵ .
- c. O estado $\{2\}$ vai para o estado $\{3\}$ na entrada b e não podemos ir mais longe a partir de 3 com setas ϵ .
- d. O estado $\{1\}$ vai para $\emptyset$ na entrada a , porque nenhuma seta a sai dele. Ele vai para $\{2\}$ na entrada b .
- e. Note que o procedimento especifica que seguimos as setas ϵ depois que cada símbolo de entrada é lido.
- f. O estado $\{3\}$ vai para $\{1, 3\}$ na entrada a , pois em $N4$ o estado 3 vai para 1 na entrada a e 1, por sua vez, vai para 3 com uma seta ϵ .
- g. O estado $\{3\}$ na entrada b vai para $\emptyset$.
- h. O estado $\{1, 2\}$ na entrada a vai para $\{2, 3\}$, pois 1 não aponta para nenhum estado com seta a e 2 aponta para ambos 2 e 3 com seta a e nenhum aponta para lugar algum com setas ϵ .
- i. O estado $\{1, 2\}$ na entrada b vai para $\{2, 3\}$.
- j. Continuar até obter o diagrama de D .



4. Simplificar o autômato obtido.

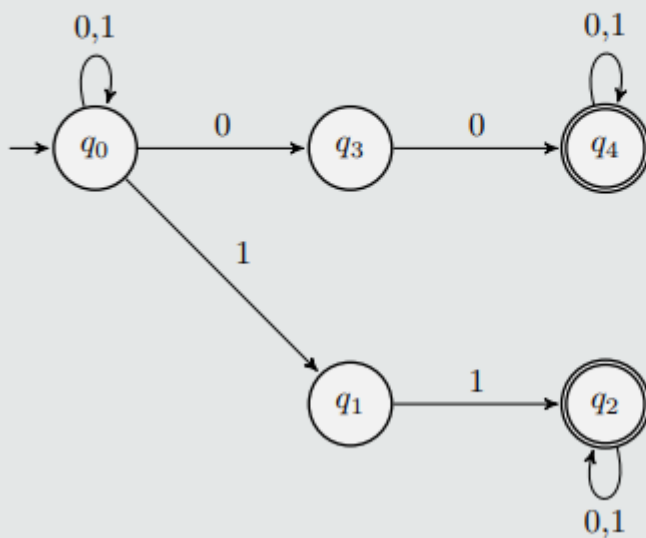
- a. **Nenhuma seta aponta para os estados $\{1\}$ e $\{1, 2\}$** , logo eles podem ser removidos sem afetar o desempenho da Máquina.



Método da Tabela de Transição

Exemplo

AFND que aceita o conjunto de todas as sentenças que contêm dois 0's ou dois 1's consecutivos.



1. Montar a tabela de transição do AFND, marcando os estados iniciais e finais

	δ	0	1
$\rightarrow$	q_0	$\{q_0, q_3\}$	$\{q_0, q_1\}$
	q_1	—	$\{q_2\}$
$\leftarrow$	q_2	$\{q_2\}$	$\{q_2\}$
	q_3	$\{q_4\}$	—
$\leftarrow$	q_4	$\{q_4\}$	$\{q_4\}$

Notação:

$\rightarrow$: marcar o estado inicial.

$\leftarrow$: marca os estados finais de aceitação.

2. **Construir a tabela de transição do AFD. Identificar o 1º estado do AFND e criar novos estados etiquetando-os com a concatenação dos estados do AFND alcançáveis pelas mesmas transições.**

- a. **Iniciando por q_0** , olhamos para a tabela de transição do AFND e selecionamos a linha q_0 e coluna 0, o valor de $\delta(q_0, 0) = \{q_0, q_3\}$, que nos devolve um conjunto dos estados alcançáveis com a entrada 0. Com os elementos do conjunto de estados, criamos um **novo estado q_0q_3** com a concatenação dos elementos do conjunto.

Tabela de Transição Original			Tabela de Transição Nova		
	δ	0	1		
$\rightarrow$	q_0	$\{q_0, q_3\}$	$\{q_0, q_1\}$	q_0	q_0q_3
	q_1	—	$\{q_2\}$	q_0q_1	

b.

- c. Criamos uma linha na tabela de transição para o novo estado q_0q_3 . E para esse novo estado, iremos **selecionar as linhas da tabela de transição original q_0 e q_3 para as entradas 0 e 1, e fazendo a união dos conjuntos selecionados em cada linha, temos os estados gerados $q_0q_3q_4$ e q_0q_1 , respectivamente.**

Tabela de Transição Nova Atualizada		
δ	0	1
q_0	q_0q_3	q_0q_1
q_0q_3	$q_0q_3q_4$	q_0q_1

d.

- e. Para cada novo estado criado, para cada símbolo do alfabeto preencher a nova tabela de transições criando novos estados quando preciso.

- i. Exemplo: para o novo estado q_0q_3 , segunda linha da nova tabela de transição, selecionamos as linhas da tabela de transição original para q_0 e q_3 , os conjuntos $\{q_0, q_3\}$ e $\{q_4\}$ são obtidos para a entrada 0, **resultando da união um novo conjunto $\{q_0, q_3, q_4\}$ que irá criar um novo estado $q_0q_3q_4$, $\delta'(q_0q_3, 0) = \{q_0, q_3, q_4\}$.**

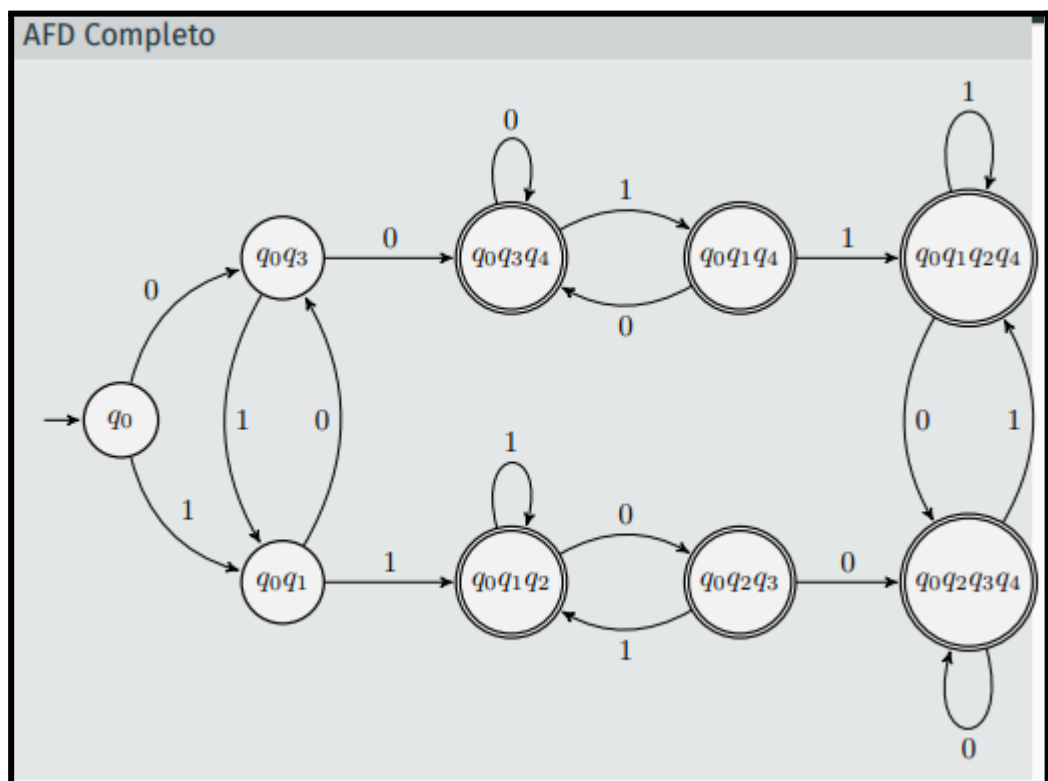
- f. Continuamos na construção da nova tabela de transição, seguindo a partir dos novos estados criados, até completarmos todos os estados.

Tabela de Transição Nova Final		
δ	0	1
q_0	q_0q_3	q_0q_1
q_0q_3	$q_0q_3q_4$	q_0q_1
q_0q_1	q_0q_3	$q_0q_1q_2$
$q_0q_3q_4$	$q_0q_3q_4$	$q_0q_1q_4$
$q_0q_1q_2$	$q_0q_2q_3$	$q_0q_1q_2$
$q_0q_1q_4$	$q_0q_3q_4$	$q_0q_1q_2q_4$
$q_0q_2q_3$	$q_0q_2q_3q_4$	$q_0q_1q_2$
$q_0q_1q_2q_4$	$q_0q_2q_3q_4$	$q_0q_1q_2q_4$
$q_0q_2q_3q_4$	$q_0q_2q_3q_4$	$q_0q_1q_2q_4$

g.

3. Todos os estados que contenham os estados finais de aceitação do AFND original, neste exemplo q_2 e q_4 , serão estados de aceitação no novo AFD.

4. Desenhar o novo AFD com base na tabela de transições.



a.

5. Simplificar o diagrama, retirando estados finais repetidos ou estados inalcançáveis. (nesse exemplo não acontece)

Tabela com E

Exemplo:

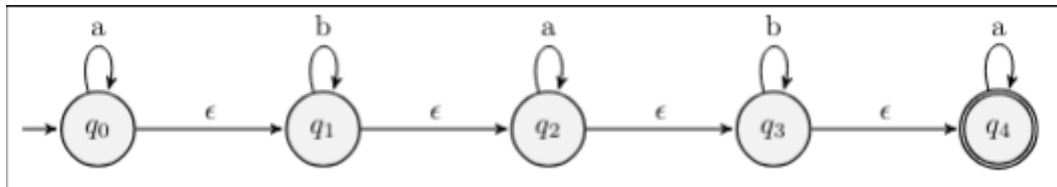


Tabela de transições do AFND

Obs: **ε-fecho é o próprio estado + tudo que dá pra alcançar só com ε**

Estado	a	b	ε	ε-fecho
→q0	{q0}	∅	{q1}	{q0, q1, q2, q3, q4}
q1	∅	{q1}	{q2}	{q1, q2, q3, q4}
q2	{q2}	∅	{q3}	{q2, q3, q4}
q3	∅	{q3}	{q4}	{q3, q4}
←q4	{q4}	∅	∅	{q4}

Tabela de Transição Nova (todos que tem q4 são aceitação)

Estado	a	b
→←q0q1q2q3q4	q0q1q2q3q4	q1q2q3q4
←q1q2q3q4	q2q3q4	q1q2q3q4
←q2q3q4	q2q3q4	q3q4
←q3q4	q4	q3q4
←q4	q4	∅
∅	∅	∅

Obs: deve pegar primeiro as transições com a/b e depois aplicar ε-fecho das transições (juntar os estados aí aplicar o ε-fecho para cada estado dessa junção e juntar esse resultado)

Ex: o b do 1º estado, normalmente seria só q1q3, mas ao aplicar o e-fecho para q1 o resultado é q1q2q3q4, e o e-fecho do q3 é q3q4!

Obs2: Só aplicamos e-fecho para estados que o e muda esse estado! Ou seja, que tem uma transição com e.

Expressões Regulares

Transformação de Expressões para Autômatos

Passo a passo:

1. Casos básicos (base da construção)

a. Símbolo simples (ex: a)

i. $(q_0) \xrightarrow{a} (q_1)$

b. ϵ (vazio)

i. $(q_0) \xrightarrow{\epsilon} (q_1)$

2. União ($A + B$)

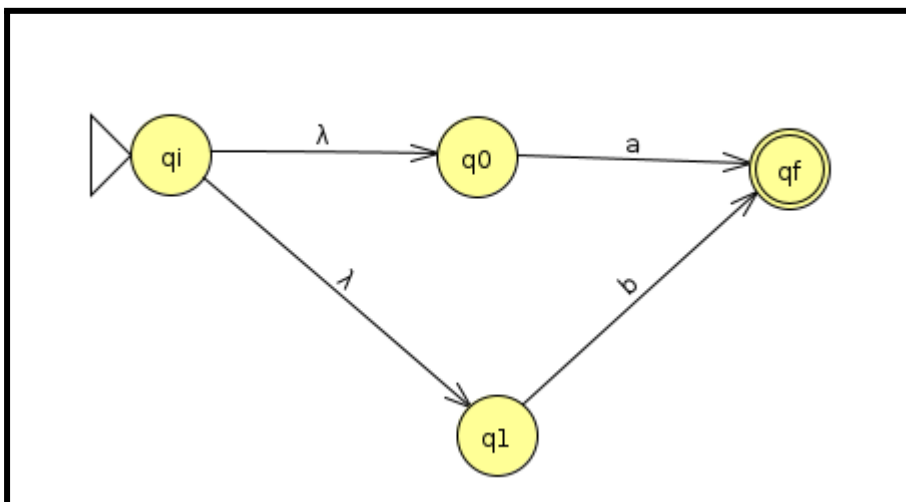
a. Representa ou A ou B

b. Construção da expressão $a \cup b$:

i. Cria um novo estado inicial (q_i)

ii. Cria um novo estado final (q_f)

iii. Usa transições ϵ

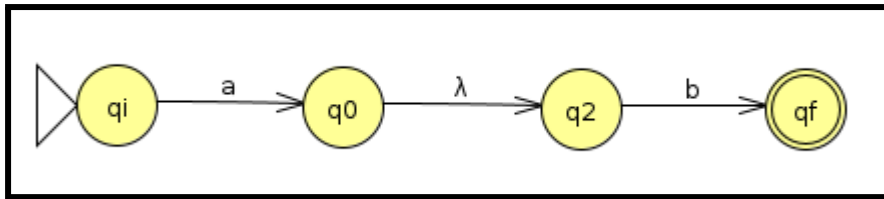


3. Concatenação (AB)

a. Representa: A seguido de B

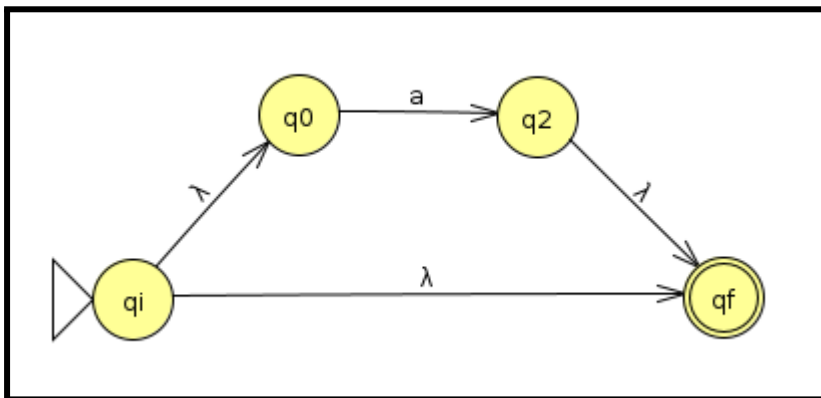
b. Construção da expressão ab :

i. Liga o final de A ao início de B com ϵ



4. Fecho de Kleene (A^*)

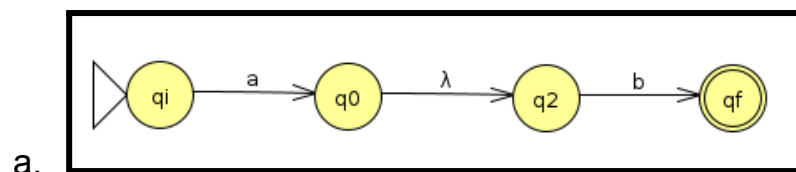
- a. Representa: **zero ou mais repetições de A**
- b. Construção de a^* :
 - i. Começa com o NFA de **$a: q_0 \xrightarrow{a} q_1$**
 - ii. Adiciona dois novos estados: **q_i (novo inicial) e q_f (novo final)**
 - iii. Conecta com ϵ -transições



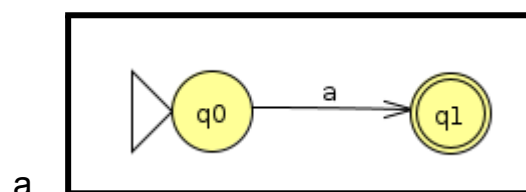
Exemplo:

Construir o autômato da expressão **$(ab \cup a)^*$**

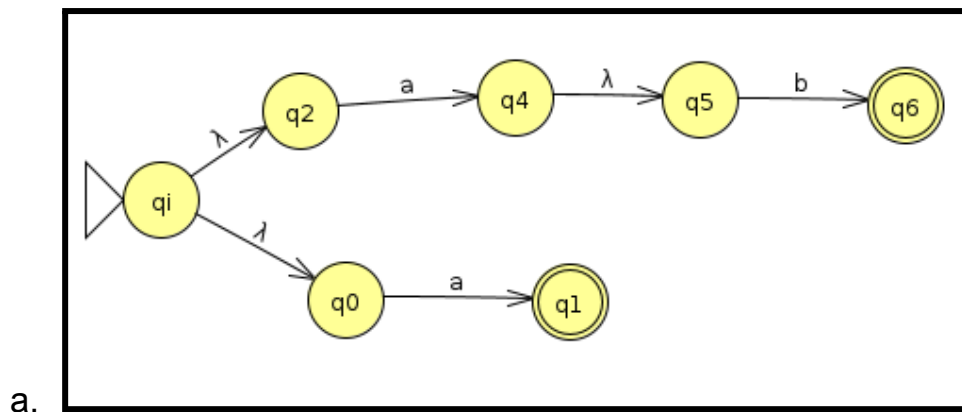
1. ab :



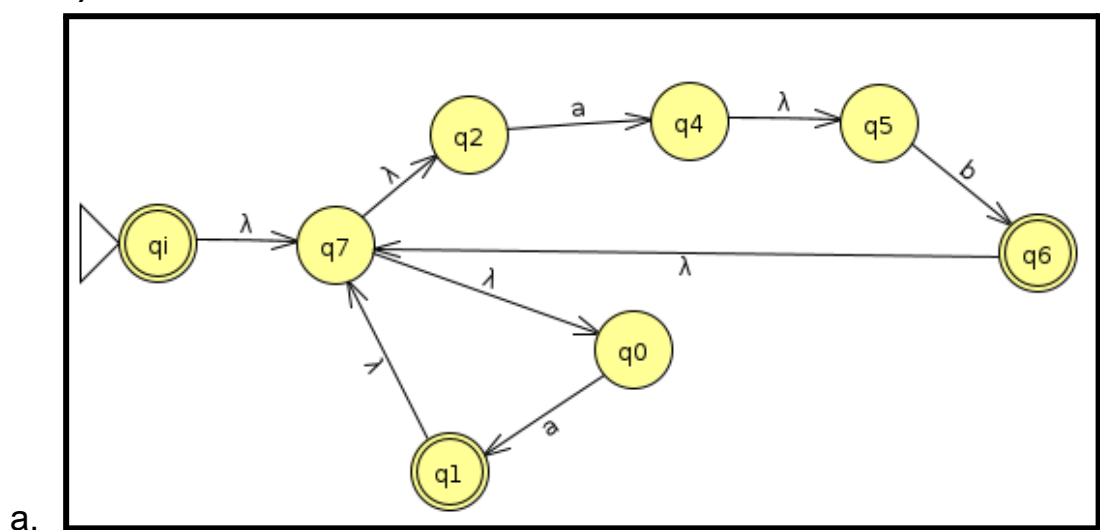
2. a :



3. $ab \cup a$



4. $(ab \cup a)^*$



Transformação de Autômatos para Expressões

A ideia é transformar o autômato em um AFND Generalizado (AFNG), onde:

- As transições não são só símbolos (a, b), mas expressões regulares
- Depois, vamos removendo estados, ajustando as transições
- No final, sobra só:
 - estado inicial novo
 - estado final novo
 e a transição entre eles será a ER final

1. Criar novos estados

a. Você adiciona:

- i. $q_{\text{início}} \rightarrow$ novo estado inicial
- ii. $q_{\text{final}} \rightarrow$ novo estado final

b. Regras:

- i. $q_{\text{início}}$ tem transição ϵ para o antigo inicial
- ii. Estados finais antigos agora vão com ϵ para q_{final}

2. Transformar transições em ER

- a. Se tinha a , vira " a "
- b. Se tinha múltiplas transições:
 - i. a ou $b \rightarrow$ vira: $a + b$
- c. Se não tinha transição $\rightarrow$ considere como $\emptyset$

3. Eliminar Estados

- a. Quando você remove um estado k , você precisa atualizar as transições:

- i. Se existe caminho:

1. $i \rightarrow k \rightarrow j$

- a. com:

- i. R_{ik} (de i para k)

- ii. R_{kk} (loop em k)

- iii. R_{kj} (de k para j)

- ii. Então você cria/atualiza:

1. $R_{ij} = R_{ij} + R_{ik} (R_{kk})^* R_{kj}$

- a. Isso significa:

- i. Caminho direto que já existia (R_{ij})

- ii. OU

- iii. Ir para $k \rightarrow$ repetir k várias vezes $\rightarrow$ sair de k

- b. Obs:

- i. Obs: no lugar de $+$ pode colocar U ou $|$

1. Significam a mesma coisa!

4. Exemplo1:

- a. Strings que começam com a e terminam com b

- b. Autômato:

- i. $q_0 \xrightarrow{a} q_1$

- ii. $q_1 \xrightarrow{b} q_2$

1. q_0 inicial, q_2 final

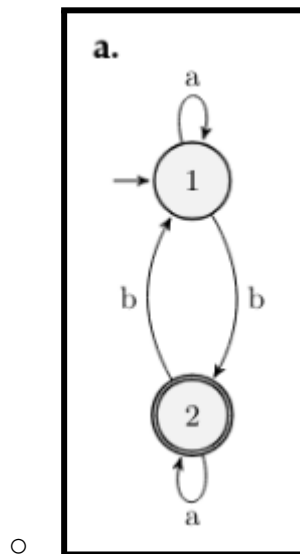
- Adicionando estados

- $q_{\text{início}} \rightarrow q_0 (\epsilon)$

- $q_2 \rightarrow q_{\text{final}} (\epsilon)$

- Eliminado estados
 - **Elimina q1 (q0→q2):**
 - $q0 \rightarrow q1 = a$
 - $q1 \rightarrow q2 = b$
 - Resultado:
 - $q0 \rightarrow q2 = ab$
 - **Elimina q0(q_i→q2):**
 - $q_inicio \rightarrow q0 = \epsilon$
 - $q0 \rightarrow q2 = ab$
 - Resultado:
 - $q_inicio \rightarrow q2 = ab$
 - **Elimina q2(q_i→q_f):**
 - $q2 \rightarrow q_final = \epsilon$
 - Resultado final:
 - **ab**

- Exemplo 2:



1. Autômato Original

- a. Estados:
 - i. Inicial: 1
 - ii. Final: 2
- b. Transições:
 - i. $1 \rightarrow 1$ com **a**
 - ii. $1 \rightarrow 2$ com **b**
 - iii. $2 \rightarrow 1$ com **b**
 - iv. $2 \rightarrow 2$ com **a**

2. Transformar em AFNG

- a. Adicionamos:
 - i. q_i (novo inicial)
 - ii. q_f (novo final)
- b. Transições novas:
 - i. $q_i \rightarrow 1$ com ϵ
 - ii. $2 \rightarrow q_f$ com ϵ

3. Escrever todas as transições como ER

- a. Agora temos:
 - i. $q_i \rightarrow 1 = \epsilon$
 - ii. $1 \rightarrow 1 = a$
 - iii. $1 \rightarrow 2 = b$
 - iv. $2 \rightarrow 1 = b$
 - v. $2 \rightarrow 2 = a$
 - vi. $2 \rightarrow q_f = \epsilon$

4. Eliminar o estado 1

- a. Vamos usar a fórmula:
 - i. $R_{ij} = R_{ij} + R_{ik} (R_{kk})^* R_{kj}$
- b. **Atualizando $q_i \rightarrow 2$ passando por 1**
 - i. Não tinha outro caminho direto (R_{ij})
 - ii. $q_i \rightarrow 1 = \epsilon$ (R_{ik}) — Chega nele
 - iii. $1 \rightarrow 1 = a$ (R_{kk})* — Loop
 - iv. $1 \rightarrow 2 = b$ (R_{kj}) — Sai dele
 - 1. Substituindo:
 - a. $q_i \rightarrow 2 = \epsilon \cdot (a)^* \cdot b$
 - 2. Resultado:
 - a. $q_i \rightarrow 2 = a^* b$
- c. **Atualizando $2 \rightarrow 2$ passando por 1**
 - i. $2 \rightarrow 1 = b$ (R_{ik})
 - ii. $1 \rightarrow 1 = a$ (R_{kk})*
 - iii. $1 \rightarrow 2 = b$ (R_{kj})
 - iv. já existe $2 \rightarrow 2 = a$ (R_{ij})
 - 1. Substituindo:
 - a. $2 \rightarrow 2 = a + b (a)^* b$

d. Agora o estado 1 foi eliminado

5. Eliminar o estado 2

a. Agora só temos:

- i. q_i
- ii. 2
- iii. q_f

b. Atualizar $q_i \rightarrow q_f$ passando por 2

- i. $q_i \rightarrow 2 = a^* b$
- ii. $2 \rightarrow 2 = a + b a^* b$
- iii. $2 \rightarrow q_f = \epsilon$

1. Substituindo:

$$a. q_i \rightarrow q_f = (a^* b) (a + b a^* b)^* \epsilon$$

c. Simplificando:

- i. ϵ no final some
- ii. resultado final:

1. $a^* b (a + b a^* b)^*$

a. Obs: no lugar de + pode colocar U ou |

Minimização de AFDs

- Minimizar um AFD (Autômato Finito Determinístico) significa:
 - **reduzir o número de estados**
 - **sem mudar a linguagem reconhecida**
- Ou seja, deixar o autômato **mais simples**, mas **equivalente**.

- A gente junta estados que são **equivalentes**.
- **Dois estados são equivalentes se:**
 - **Para qualquer entrada, eles se comportam da mesma forma**
 - **Não dá pra distinguir eles** (mesma aceitação/rejeição)

Passo a passo:

1. Transformar o autômato em AFD

- a. Pode haver casos que falta uma trap, não esquecer de colocar ela antes dos próximos passos!

2. Remover estados inalcançáveis

- a. Estados que você nunca chega a partir do inicial.

3. Separar estados em grupos

a. Primeira divisão:

- i. **Finais (F)**
- ii. **Não finais (N)**

1. Exemplo:

- a. $F = \{q2, q3\}$
- b. $N = \{q0, q1\}$

4. Verificar Equivalência

a. **Ver se dentro de um grupo existem estados que se comportam diferente.**

b. Para cada símbolo do alfabeto:

- i. **Veja para onde cada estado vai**
- ii. **Veja em qual grupo ele cai**
- iii. **Se forem diferentes → separa**

c. Exemplo:

Estado	a	b	Final?
q0	q1	q2	✗
q1	q0	q3	✗
q2	q3	q0	✓
q3	q2	q1	✓

Separar finais e não finais:

$G1 = \{q2, q3\}$ (finais)

$G2 = \{q0, q1\}$ (não finais)

Verificar equivalência:

Grupo {q0,q1}

- q0:
 - $a \rightarrow q1$ (grupo N)
 - $b \rightarrow q2$ (grupo F)
- q1:
 - $a \rightarrow q0$ (grupo N)
 - $b \rightarrow q3$ (grupo F)
- **Mesmo padrão: (N,F)**

- q_0 e q_1 são equivalentes

Grupo $\{q_2, q_3\}$

- q_2 :
 - $a \rightarrow q_3$ (F)
 - $b \rightarrow q_0$ (N)
- q_3 :
 - $a \rightarrow q_2$ (F)
 - $b \rightarrow q_1$ (N)
- Mesmo padrão: (F, N)
- q_2 e q_3 são equivalentes

Grupos finais: $[q_0, q_1]$ e $[q_2, q_3]$

Cada grupo vira um estado novo:

Novo estado	Representa
A	$\{q_0, q_1\}$
B	$\{q_2, q_3\}$

Obs: não precisa colocar essa de A ou B, q_0q_1 e q_2q_3 já estão certos!

Transições:

- A:
 - $a \rightarrow A$
 - $b \rightarrow B$
- B:
 - $a \rightarrow B$
 - $b \rightarrow A$

Estado final: B (porque contém estados finais)

Autômatos com Saída

- **Autômatos Finitos** possui aplicações práticas restritas, pois a saída é limitada à informação de aceita/rejeita, indicando se reconhece ou não a entrada como palavra pertencente à linguagem.
- Assim, a definição de **Autômato Finito** é estendida prevendo **saídas associadas aos estados (Máquina de Moore)** ou às **transições (Máquina de Mealy)**.

- Sem alterar a classe de linguagens reconhecidas, é possível estender a definição de autômato finito, incluindo-se a geração de uma palavra de saída.
- Em ambas as máquinas, a saída não pode ser lida, ou seja, **a saída não pode ser usada como memória auxiliar.**
- **A saída é definida sobre um alfabeto especial, denominado alfabeto de símbolos de saída, o qual pode ser igual ao alfabeto de entrada.**
- A saída é armazenada em uma fita de saída independente da de entrada.
- A cabeça da fita de saída move uma célula para a direita a cada símbolo gravado.
- O resultado do processamento do autômato finito é seu estado final (condição de aceita/rejeita) e a informação contida na fita de saída.
- As máquinas de Mealy e Moore são modificações sobre o autômato finito determinístico. A extensão da definição, prevendo facilidades de não determinismo e movimentos vazios, é sugerida como exercícios.

Máquina de Mealy

- A **Máquina de Mealy** é um **autômato finito modificado** de forma a **gerar uma palavra de saída (a qual pode ser vazia) para cada transição da máquina.**
- Uma **máquina de Mealy** M é um **Autômato Finito Determinístico com saídas associadas às transições**. É representada por uma 6-tupla:
 - $M = (Q, \Sigma, \delta, q_0, F, \Delta)$, na qual:
 - Q é um **conjunto de estados possíveis do autômato finito.**
 - Σ é o **alfabeto de símbolos de entrada.**
 - δ é uma **função de transição** (função programa):
 - $\delta : Q \times \Sigma \rightarrow Q \times \Delta^*$
 - q_0 é **estado inicial.**
 - F é o **conjunto de estados Finais/Aceitação.**
 - Δ é um **alfabeto de símbolos de saída ou alfabeto de saída.**
 - **Bom lembrar que a saída está nas transições**, então ex:
 - de $q_0 \rightarrow q_1$, na seta estaria tipo **a/1**, pq ele muda com a e a saída é 1.

- As componentes Σ , Q , q_0 e F são como no autômato finito determinístico.
- A função programa pode ser representada como um diagrama como nos autômatos finitos, **adicionando**, na etiqueta/rótulo de cada **transição**, a **saída associada**, quando diferente da palavra vazia.
- A computação de uma Máquina de Mealy, para uma **palavra de entrada** w , **consiste na sucessiva aplicação da função programa para cada símbolo de** w (da esquerda para a direita) **até ocorrer uma condição de parada**.
- A palavra vazia como saída da função de transição indica que nenhuma gravação é realizada e obviamente, não move a cabeça da fita de saída.
- Se todas as transições geram saída vazia, então a máquina de Mealy processa como se fosse uma AF.

EXEMPLO:

Considere a Máquina de Mealy MM , cuja definição segue

$MM = (Q, \Sigma, \delta, q_0, F, \Delta)$ e $\delta : Q \times \Sigma \rightarrow Q \times \Delta^*$:

$MM = (\{q_0, q_1, q_2\}, \{a, b, c\}, \{(q_0, a) \rightarrow (q_0, 1), (q_0, b) \rightarrow (q_1, \epsilon), (q_0, c) \rightarrow (q_2, 2), (q_1, a) \rightarrow (q_0, 1), (q_1, b) \rightarrow (q_1, \epsilon), (q_1, c) \rightarrow (q_2, 2), (q_2, a) \rightarrow (q_0, 1), (q_2, b) \rightarrow (q_1, \epsilon), (q_2, c) \rightarrow (q_2, 2)\}, q_0, \{q_2\}, \{1, 2\})$.

E as sentenças **aaabbbccc** e **aabbccaabb**. Quais são as saídas geradas por T para cada sentença e quais das sentenças são aceitas por T ?

Tabela de transições

Estado	Entrada	Próx. estado	Saída
q_0	a	q_0	1
q_0	b	q_1	ϵ
q_0	c	q_2	2
q_1	a	q_0	1
q_1	b	q_1	ϵ
q_1	c	q_2	2
q_2	a	q_0	1

q_2	b	q_1	ϵ
q_2	c	q_2	2

Processamento de **aaabbbccc**

Passo	Estado	Símbolo	Próx. estado	Saída
1	q_0	a	q_0	1
2	q_0	a	q_0	1
3	q_0	a	q_0	1
4	q_0	b	q_1	ϵ
5	q_1	b	q_1	ϵ
6	q_1	b	q_1	ϵ
7	q_1	c	q_2	2
8	q_2	c	q_2	2
9	q_2	c	q_2	2

Saída: **111222** | Estado final: $q_2 \in F \rightarrow$  Aceita (parou em q_2 que é estado final então aceita!)

Processamento de **aabbccaabb**

Passo	Estado	Símbolo	Próx. estado	Saída
1	q_0	a	q_0	1
2	q_0	a	q_0	1
3	q_0	b	q_1	ϵ
4	q_1	b	q_1	ϵ
5	q_1	c	q_2	2
6	q_2	c	q_2	2
7	q_2	a	q_0	1
8	q_0	a	q_0	1

9	q_0	b	q_1	ϵ
10	q_1	b	q_1	ϵ

Saída: 112211 | Estado final: $q_1 \notin F \rightarrow \text{Rejeita}$

Máquina de Moore

- A Máquina de Moore possui uma segunda função, que gera uma palavra de saída (a qual pode ser vazia) para cada estado da máquina.
- Uma Máquina de Moore M é um autômato finito determinístico com saídas associadas aos estados. É representado por uma 7-tupla:
- $M = (Q, \Sigma, \delta, q_0, F, \Delta, \delta S)$
 - Na qual:
 - Σ é o alfabeto de entrada.
 - Q é o conjunto de estados possíveis.
 - δ é uma função programa ou função de transição: $\delta : Q \times \Sigma \rightarrow Q$ (função parcial: cada elemento do domínio está relacionado com, no máximo, um elemento do codomínio).
 - q_0 é o estado inicial.
 - F é o conjunto de estados finais.
 - Δ é um alfabeto de símbolos de saída.
 - δS é uma função de saída: $\delta S : Q \rightarrow \Delta^*$ (função total: para todo $a \in A$ existe $b \in B$ tal que $f(a) = b$. É uma função parcial definida para todos os elementos.)

Anotar exemplo da máquina de moore

Gramática Regular

Gramática

- Linguagem de programação
 - definida pelo conjunto de todos os programas (palavras)
- Linguagem de propósitos gerais como Pascal

- **conjunto** de todos os **programas** é **infinito**
- não é definição adequada para implementação em computador
- **Formalismo Gramática**
 - uma maneira de **especificar** de **forma finita** **linguagens** **(eventualmente) infinitas**
- **Gramática é, basicamente**
 - conjunto **finito** de **regras**
 - quando **aplicadas sucessivamente**, geram **palavras**
 - conjunto de todas as **palavras geradas** por uma gramática
 - define a **linguagem**
- **Gramática de Chomsky, Gramática Irrestrita ou Gramática**
 - **$G = (V, T, P, S)$**
 - **V**, conjunto **finito** de símbolos **variáveis** ou **não-terminais**
 - **T**, conjunto **finito** de símbolos **terminais** disjunto de **V**
 - **$P: (V \cup T)^+ \rightarrow (V \cup T)^*$** , relação **finita**: **Produções**
 - par da relação: **regra de produção** ou **produção**
 - **S**, elemento distinguido de **V**: **símbolo inicial** ou **variável inicial**
 - **Representação** de uma regra de produção **(α, β)**
 - **$\alpha \rightarrow \beta$**
 - **Representação abreviada** para **$\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$**
 - **$\alpha \rightarrow \beta_1 \ \beta_2 \ \dots \ \beta_n$**

Autômatos com Pilha

- Um **Autômato com Pilha (AP)** é um Autômato Finito que possui uma **memória extra chamada pilha**.
- A pilha funciona segundo a regra:
 - **LIFO (Last In, First Out)**
- Exemplo:

Empilha A

Empilha B

Empilha C

Pilha:

C <- topo

B

A

Desempilha -> sai C

Desempilha -> sai B

Desempilha -> sai A

Por que a pilha é importante?

- Um AFD não consegue reconhecer:
$$L = \{0^n 1^n \mid n \geq 0\}$$
- porque ele teria que lembrar quantos zeros apareceram.
- A pilha resolve isso.
- Exemplo:
 - Entrada: 000111
 - Leitura dos zeros:
 - 0 -> empilha X
 - 0 -> empilha X
 - 0 -> empilha X
 - Pilha:
 - X
 - X
 - X
 - Z
 - (Z = símbolo inicial)
 - Agora aparecem os 1's:
 - 1 -> desempilha X
 - 1 -> desempilha X
 - 1 -> desempilha X
 - Quando a entrada acabar e restar apenas Z:
 - aceita

Operações da Pilha

Inicializar a Pilha

- Quando começamos o AP, normalmente colocamos o marcador de fundo \$.

- Transição: $\epsilon, \epsilon \rightarrow \$$
- Significado:
 - não lê nada da entrada
 - não remove nada da pilha
 - empilha $\$$

- Resultado:

Pilha antes:
(vazia)

Pilha depois:
 $\$$

Empilhar

- Suponha que queremos **guardar um 0 para cada 0 lido**.
- Transição: $0, \epsilon \rightarrow 0$
- Significado:
 - lê um 0
 - não remove nada da pilha
 - empilha um 0

Exemplo:

None

Pilha antes:

$\$$

Lê 0

Pilha depois:

0

$\$$

Lendo outro 0:

None

Pilha antes:

0

\$

Lê 0

Pilha depois:

0

0

\$

Desempilhar

- Transição: **1, 0 $\rightarrow \epsilon$**
- Significado:
 - lê um 1
 - remove um 0 do topo
 - não coloca nada

Exemplo:

None

Pilha antes:

0

0

\$

Lê 1

Pilha depois:

0

\$

Trocar um Símbolo por Outro

- Às vezes queremos substituir.
- Exemplo: **a, X $\rightarrow$ Y**

- Significa:
 - **lê um a**
 - **remove X**
 - **coloca Y**

Resultado:

None

Antes:

X

\$

Depois:

Y

\$

Não Fazer Nada

- Às vezes queremos apenas ler a entrada.
- Exemplo: **a, X → X**
- Significa:
 - **lê um a**
 - **remove X**
 - **recoloca X**
- Ou seja, a pilha permanece igual.

Formas de Aceitação

- Existem duas:
 - **Por estado final: chega num estado final.**
 - **Por pilha vazia: a pilha fica vazia.**

Padrões Clássicos

Igual Quantidade

$0^n 1^n$

Estratégia:

- **empilha para cada 0**

- **desempilha para cada 1**

Palíndromos

ww^R

Exemplo:

011110

Estratégia:

- **empilha primeira metade**
- adivinha (não determinismo) onde é o meio
- **desempilha comparando**

Balanceamento

Parênteses:

(() ())

Estratégia:

- ($\rightarrow$ **empilha**
-) $\rightarrow$ **desempilha**

No final: pilha vazia

Características

- Os APs são frequentemente **não determinísticos**.
- Isso significa que eles podem:
 - escolher um caminho
 - escolher outro caminho
- e **basta um deles aceitar**.
- Isso será essencial em:
 - **palíndromos**
 - **linguagens com OU ($\vee$)**

Gramáticas Livre de Contexto

Uma GLC é uma tupla:

$$G=(V,\Sigma,P,S)$$

onde:

- **V = conjunto de variáveis (não-terminais)**
- **Σ = conjunto de terminais**

- **P = conjunto de produções**
- **S = símbolo inicial**

Exemplo:

$$S \rightarrow aSb \mid \epsilon$$

Aqui:

- **Variável: S**
- **Terminais: a,b**
- **Símbolo inicial: S**

Essa gramática gera:

ϵ , ab, aabb, aaabbb,...

Derivação

- **Aplicamos produções até obter apenas terminais.**
- Exemplo:

$$S \Rightarrow aSb$$

$$\Rightarrow aaSbb$$

$$\Rightarrow aa\epsilon bb$$

$$\Rightarrow aabb$$

Produções Vazias

- **São produções do tipo:**

$$A \rightarrow \epsilon$$

Exemplo:

$$S \rightarrow aAb$$

$$A \rightarrow c \mid \epsilon$$

Como A pode desaparecer:

$$S \rightarrow acb$$

ou

$$S \rightarrow ab$$

Variáveis Anuláveis

- **Uma variável é anulável se consegue gerar ϵ .**

Exemplo:

$$A \rightarrow \epsilon$$

- **Logo A é anulável.**

Exemplo mais difícil:

$$A \rightarrow BC$$

$$B \rightarrow \epsilon$$

$$C \rightarrow \epsilon$$

Então:

$$A \Rightarrow BC \Rightarrow \epsilon \epsilon \Rightarrow \epsilon$$

Logo A também é anulável.

Eliminação de Produções Vazias

1. Encontrar anuláveis

Exemplo:

$$S \rightarrow AB$$

$$A \rightarrow a \mid \epsilon$$

$$B \rightarrow b$$

A é anulável.

2. Criar alternativas

Como A pode sumir:

$$S \rightarrow AB$$

gera

$$S \rightarrow AB \mid B$$

3. Remover $A \rightarrow \epsilon$

Resultado:

$$S \rightarrow AB \mid B$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Produções Unitárias

- São produções do tipo:

$$A \rightarrow B$$

- somente uma variável.

Exemplo:

$$S \rightarrow A$$

$$A \rightarrow B$$

$$B \rightarrow a$$

Substituímos:

$$S \rightarrow a$$

$$A \rightarrow a$$

e removemos as unitárias.

Símbolos Inúteis

- Existem dois tipos.

Tipo 1: Não Geradores

- **Nunca conseguem produzir terminais.**

Exemplo:

$$A \rightarrow BC$$
$$B \rightarrow AB$$
$$C \rightarrow B$$

Nunca aparece terminal.

Logo: A, B, C são inúteis.

Tipo 2: Inacessíveis

- **Não podem ser alcançados a partir de S.**

Exemplo:

$$S \rightarrow aA$$
$$A \rightarrow b$$
$$B \rightarrow c$$

B nunca é usado.

Logo removemos B.

Ordem Correta da Simplificação

1. Remover ϵ -produções
2. Remover unitárias
3. Remover símbolos inúteis

Forma Normal

- Uma **Forma Normal** é uma **versão "padronizada"** de uma Gramática Livre de Contexto (GLC).
- A ideia é:
 - **simplificar a gramática;**
 - **facilitar provas teóricas;**
 - **permitir algoritmos de reconhecimento (como CYK).**
- As duas formas normais mais importantes são:
 1. **Forma Normal de Chomsky (FNC)**
 2. **Forma Normal de Greibach (FNG)**

Forma Normal de Chomsky (FNC)

- Uma gramática está em FNC quando **todas as produções** possuem um destes formatos:

- **Tipo 1:**

$$A \rightarrow BC$$

- onde:

- **A, B, C** são variáveis
- **B e C** não são o símbolo inicial

- Exemplo:

- $S \rightarrow AB$

- **Tipo 2:**

$$A \rightarrow a$$

- onde:

- **a** é terminal

- Exemplo:

- $A \rightarrow a$

- **Exceção**

- Se a linguagem contém ϵ :

$$S \rightarrow \epsilon$$

- pode existir.

O que não pode existir na FNC

1. Produções ϵ

Não pode:

$$A \rightarrow \epsilon$$

(exceto o símbolo inicial)

2. Produções unitárias

Não pode:

$$A \rightarrow BA$$

3. Mistura terminal + variável

Não pode:

$$A \rightarrow aBA$$

4. Mais de duas variáveis

Não pode:

$$A \rightarrow BCD$$

Passo a Passo

1. Criar novo símbolo inicial (quando necessário)

- Se o símbolo inicial aparece do lado direito:

$$S_0 \rightarrow S$$

2. Remover ϵ -produções

- Encontrar variáveis anuláveis.

- Exemplo:

$$A \rightarrow \epsilon$$

- Gerar todas as combinações removendo as ocorrências da variável anulável.

- Exemplo:

$$S \rightarrow ABA$$

- com A anulável:

$$S \rightarrow ABA \mid BA \mid AB \mid BS$$

3. Remover produções unitárias

- Trocar:

$$A \rightarrow B$$

- pelas produções de B.

- Exemplo:

$$A \rightarrow B$$

$$B \rightarrow a \mid CD$$

- vira:

$$A \rightarrow a \mid CD$$

4. Remover símbolos inúteis

- Não geradores: não produzem terminais.

- Inacessíveis: Não podem ser alcançados a partir do símbolo inicial.

5. Tratar terminais em produções longas

- Se aparecer:

- b. crie: $A \rightarrow aB$
- c. e substitua: $X \rightarrow a$
- d. Exemplo: $A \rightarrow XB$
- e. vira: $A \rightarrow aBC$
- $X \rightarrow a$
 $A \rightarrow XBC$

6. Quebrar produções longas

- a. Se houver: $A \rightarrow BCD$
- b. crie: $X \rightarrow CD$
- c. e substitua: $A \rightarrow BX$
- d. Se houver: $A \rightarrow BCDE$
- e. faça: $X_1 \rightarrow DE$
 $X_2 \rightarrow CX_1$
 $A \rightarrow BX_2$

Máquinas de Turing

Definição

- Uma **Máquina de Turing** é composta por:
 - **Uma fita** (teoricamente infinita)
 - **Um cabeçote** de leitura/escrita
 - **Um conjunto de estados**
 - **Uma função de transição**

Estrutura da Fita

- Imagine:

None

□ □ □ **a** **b** **b** **a** □ □ □
 ^

- O cabeçote está lendo o símbolo **a**.
- A máquina pode:
 - **Ler**
 - **Escrever**
 - **Mover para direita (R)**
 - **Mover para esquerda (L)**

Função de Transição

- Num AFD:
 $\delta(q,a)=q_1$
- Na MT:
 $\delta(q,a)=(p,b,R)$
- Significa:
 - **Se estou em q lendo a,**
 - **escrevo b,**
 - **vou para p,**
 - **movo para direita.** (R de right)

Exemplo:

Reconhecer:

$L=\{a\}$

Transições:

None

$\delta(q_0, a)=(q_1, a, R)$

e q_1 é estado de aceitação.

Entrada:

None

a

Execução:

```
None
q0 lendo a
↓
q1
aceita
```

Linguagens Recursivas

Linguagem Recursivamente Enumerável

- Uma linguagem é **RE** quando **existe uma MT** que:
 - **aceita todas as palavras da linguagem**
- **mas**
 - **pode ficar em loop para palavras fora dela.**

Exemplo:

Linguagem:

$$L = \{ww \mid w \in \{0,1\}^*\}$$

Existe MT que tenta verificar.

Se a palavra estiver na linguagem:

```
None
aceita
```

Se não:

```
None
pode rejeitar
ou ficar em loop
```

Linguagem Recursiva

- **Existe uma MT que:**
 - **aceita palavras da linguagem**
 - **rejeita palavras fora dela**
- **e sempre para.**

Tipo	Aceita	Rejeita	Pode entrar em loop
RE	sim	talvez	sim
Recursiva	sim	sim	não

MT Reconhecedoras e Decisoras

Reconhecedor

- Uma **MT reconhece L** se:
 - **aceita toda palavra de L**
- **mas para palavras fora de L:**
 - **pode rejeitar**
 - **pode entrar em loop**

Exemplo:

$w \in L$

→ aceita

$w \notin L$

→ rejeita ou loop

Decisor

- Uma **MT decide L** se:
 - **aceita palavras de L**
 - **rejeita palavras fora de L**
 - **sempre termina**

Exemplo:

None

entrada = 0011

aceita

```
entrada = 0101
```

```
rejeita
```

e nunca entra em loop.

MT como Processadora de Funções

Exemplo

Função:

$$f(n)=n+1$$

Representação unária:

None

```
111
```

significa 3.

Resultado esperado:

None

```
1111
```

significa 4.

Como a MT faz?

Entrada:

None

```
111□
```

Vai andando para direita:

None

```
111□
```

```
  ^
```

Quando encontra branco:

None

□ → 1

Resultado:

None

1111

A máquina não precisa responder:

None

aceita/rejeita

Ela pode produzir uma nova palavra na fita.

Modelos Equivalentes